

Otimização de Entregas com K-means: Geoclusterização para Posicionamento de Centros de Distribuição

Fábio do Nascimento Patão, Isabela da Silva Araujo Veiga

Instituto de Computação – UFRJ
Trabalho da disciplina – 2026

Resumo. Este trabalho apresenta um sistema de otimização de rotas de entrega baseado no algoritmo de agrupamento K-means. Dado um conjunto de pontos geográficos de entrega, a aplicação os agrupa em clusters e estima a localização ideal para um centro de distribuição (CD) dentro de cada grupo. O centróide produzido pelo K-means, que pode cair em terreno não navegável, como rios ou lagos, é corrigido por meio da API OSRM (Open Source Routing Machine), que "encaixa" o ponto no local mais próximo da malha viária real. A solução foi implementada como uma aplicação web interativa utilizando Streamlit, permitindo que o usuário adicione pontos de entrega clicando em um mapa e visualize em tempo real os clusters e os centros de distribuição resultantes.

1 Introdução

Empresas de logística e comércio eletrônico – como grandes redes de varejo e marketplaces – enfrentam constantemente o desafio de atender clientes espalhados por toda uma cidade ou região. Um dos principais fatores de custo nesse cenário é a distância entre os centros de distribuição (CDs) e os pontos de entrega: quanto maior a distância média percorrida pelos veículos, maior o consumo de combustível, o tempo de entrega e o custo operacional.

O problema central deste trabalho pode ser resumido da seguinte forma: *dado um conjunto de pontos de demanda (endereços de clientes) espalhados geograficamente, onde devem ser posicionados os centros de distribuição de modo a minimizar a distância média até os pontos que cada um deles atende?*

Este é, essencialmente, um problema de agrupamento (*clustering*) espacial, no qual cada cluster representa uma região de atendimento e o centro de cada cluster indica a localização recomendada para um CD. Optou-se por aplicar o algoritmo **K-means**, estudado na disciplina, a dados geográficos reais (coordenadas de latitude e longitude), desenvolvendo uma ferramenta interativa que permite simular esse cenário na prática.

2 Fundamentação Teórica

O K-means é um algoritmo de aprendizado não supervisionado que particiona um conjunto de n observações em k clusters, de modo a minimizar a soma das distâncias quadráticas entre cada ponto e o centróide do cluster ao qual pertence (a chamada inércia, ou *within-cluster sum of squares*). O algoritmo é iterativo: (i) k centróides iniciais são escolhidos; (ii) cada ponto é atribuído ao centróide mais próximo; (iii) os centróides são

recalculados como a média dos pontos de cada cluster; (iv) os passos (ii) e (iii) se repetem até a convergência.

No contexto deste trabalho, cada observação corresponde a um par de coordenadas geográficas (latitude, longitude) de um ponto de entrega, e o centróide resultante de cada cluster representa, em primeira aproximação, o ponto médio geográfico das entregas daquele grupo – um bom candidato à localização de um centro de distribuição.

Uma limitação relevante do K-means aplicado diretamente a coordenadas geográficas é que o centróide é uma média aritmética simples das posições, sem qualquer conhecimento sobre a malha viária real. Isso significa que o ponto resultante pode, por exemplo, cair no meio de um rio, de uma baía ou de uma área sem infraestrutura viária – um problema relevante em uma cidade como o Rio de Janeiro, cortada por baías, lagoas e morros. Essa limitação motivou a etapa de pós-processamento descrita na Seção 3.2.

3 Solução Proposta

3.1 Visão geral da aplicação

Foi desenvolvida uma aplicação web interativa, utilizando o *framework* **Streamlit**, que permite ao usuário simular o problema de posicionamento de centros de distribuição de ponta a ponta. O fluxo de uso é o seguinte:

1. o usuário clica sobre um mapa interativo (renderizado com a biblioteca **Folium**) para inserir pontos de demanda, simulando endereços de clientes;
2. o usuário escolhe, por meio de um controle deslizante, o número de centros de distribuição (k) desejado;
3. a aplicação executa o K-means (via **scikit-learn**) sobre os pontos inseridos e recalcula os clusters automaticamente a cada novo ponto;
4. os centróides resultantes são ajustados para a via mais próxima, por meio de uma chamada à API pública **OSRM**, e exibidos no mapa como ícones de centro de distribuição;
5. os pontos de entrega são coloridos de acordo com o cluster ao qual pertencem, permitindo a visualização imediata das áreas de atendimento.

A Figura 1 ilustra o resultado da aplicação: os pontos coloridos representam clientes agrupados em três regiões da Cidade Universitária (Ilha do Fundão) e arredores, no Rio de Janeiro, e os ícones de caminhão marcam os centros de distribuição sugeridos para cada grupo.

3.2 Ajuste do centróide à malha viária

Como discutido na Seção 2, a média geográfica calculada pelo K-means pode recair sobre um ponto não navegável (água, por exemplo). Para corrigir esse problema, cada centróide é enviado à API de roteamento **OSRM** (*Open Source Routing Machine*), no serviço `nearest`, que retorna o ponto da malha viária real mais próximo da coordenada informada. Dessa forma, o centro de distribuição sugerido é sempre um local acessível por veículos, e não apenas um ponto matemático abstrato. Esse ajuste é implementado pela função `snap_to_road`, cujos resultados são armazenados em cache (`st.cache_data`) para evitar chamadas repetidas à API para as mesmas coordenadas.

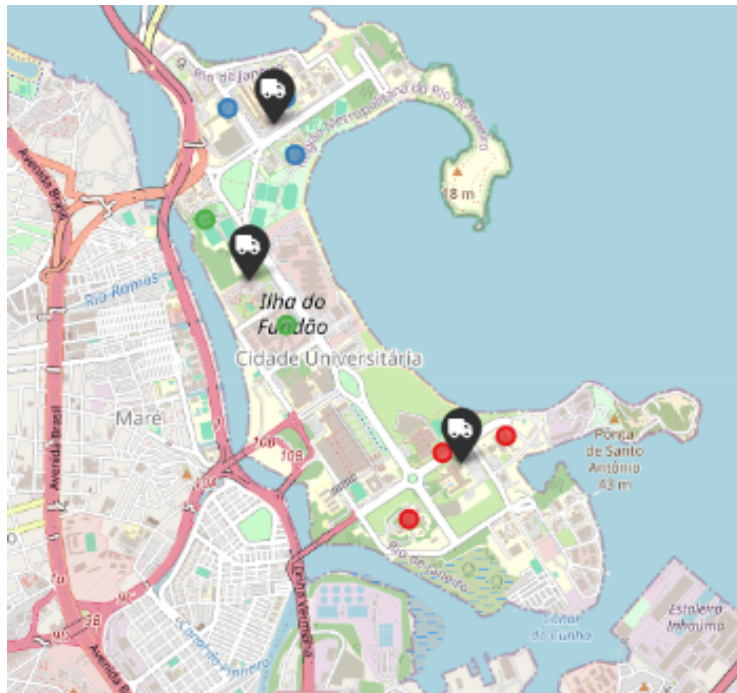


Figure 1: Aplicação em funcionamento: pontos de entrega agrupados por cor de cluster e centros de distribuição sugeridos (ícones de caminhão) na região da Ilha do Fundão, Rio de Janeiro.

3.3 Arquitetura e principais decisões de implementação

A aplicação foi construída em Python, com as seguintes bibliotecas principais:

- **Streamlit**: interface web e gerenciamento de estado da sessão (`session_state`), responsável por manter a lista de pontos de entrega entre interações do usuário;
- **Folium** e **streamlit-folium**: renderização do mapa interativo e captura de cliques do usuário;
- **scikit-learn** (KMeans): execução do algoritmo de clusterização;
- **requests**: comunicação com a API OSRM para o ajuste dos centróides.

Alguns pontos de destaque na implementação (arquivo `app.py`, reproduzido parcialmente na Figura 2):

- o número de clusters k é limitado ao intervalo de 1 a 5 e só é aplicado quando o número de pontos inseridos é maior que k , evitando erros do algoritmo com poucos dados;
- cada cluster é associado a uma cor fixa dentre uma paleta de cinco cores, tanto para os pontos de entrega quanto, implicitamente, para o centro correspondente;
- um mecanismo simples de *debounce* evita que o mesmo clique seja processado mais de uma vez, comparando a coordenada do clique atual com a do último clique registrado;
- a cada novo ponto adicionado, a aplicação recalcula os clusters e centros automaticamente (`st.rerun()`), sem exigir nenhuma ação adicional do usuário.

```

1 def get_clusters(locations, k):
2     if len(locations) <= k:
3         return None, None
4
5     df = pd.DataFrame(locations, columns=['lat', 'lon'])
6     model = KMeans(n_clusters=k, n_init=10, random_state=42)
7     model.fit(df)
8
9     centers = [snap_to_road(lat, lon) for lat, lon in model.
10                cluster_centers_]
11     return model.labels_, centers

```

Figure 2: Trecho central da aplicação: cálculo dos clusters com K-means e ajuste dos centros à malha viária.

4 Resultados e Discussão

A aplicação permite explorar de forma prática o efeito do número de clusters k sobre o posicionamento dos centros de distribuição. Ao aumentar k , os grupos ficam menores e mais concentrados geograficamente, reduzindo a distância média entre cada CD e os pontos que ele atende, porém aumentando o número de centros a serem operados – um compromisso clássico entre custo de transporte e custo de instalação/operação de novos CDs, que na prática precisaria ser avaliado junto a custos reais de operação de cada depósito.

O uso da API OSRM para ajustar os centróides mostrou-se importante mesmo em uma área geograficamente pequena: como a Ilha do Fundão e seu entorno são cercados por corpos d’água, um centróide puramente matemático facilmente cairia sobre a Baía de Guanabara, tornando a sugestão inutilizável. O ajuste à via mais próxima resolve esse problema, embora ainda não garanta que o local seja de fato apropriado para a instalação de um centro de distribuição (por exemplo, pode recair em uma via estreita ou em área residencial), o que aponta para trabalhos futuros.

Outra observação relevante é que o K-means, por utilizar distância euclidiana sobre coordenadas geográficas, não leva em conta a topologia real das ruas, rios e pontes da cidade. Duas regiões próximas em linha reta podem estar, na prática, distantes uma da outra pela malha viária (por exemplo, separadas por um braço de mar). Essa é uma limitação conhecida da aplicação direta do K-means a dados geoespaciais, e sua mitigação (por exemplo, com métricas de distância viária) é discutida na próxima seção.

5 Conclusão e Trabalhos Futuros

Este trabalho demonstrou a aplicação prática do algoritmo K-means, estudado na disciplina, à solução de um problema real de logística: o posicionamento de centros de distribuição a partir de um conjunto de pontos de entrega. Foi desenvolvida uma aplicação interativa que permite simular esse cenário, complementando o K-means com uma etapa de ajuste dos centróides à malha viária real por meio da API OSRM, resolvendo o problema prático de centróides caindo em áreas não navegáveis.

Como trabalhos futuros, destacam-se:

- aplicação do método do cotovelo (*elbow method*) ou do coeficiente de silhueta para sugerir automaticamente um valor adequado de k , em vez de exigir que o usuário o

escolha manualmente;

- substituição da distância euclidiana pela distância real de deslocamento (obtida via API de roteamento) como métrica de agrupamento, o que tornaria os clusters mais fiéis à realidade viária da cidade;
- ponderação dos pontos de entrega por volume ou frequência de demanda, de modo que clientes com maior volume de pedidos tenham peso maior na definição do centróide;
- integração com uma API de geocodificação, permitindo que o usuário insira endereços em texto em vez de clicar diretamente no mapa.

References

- [1] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, v. 1, p. 281–297.
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830.
- [3] Project OSRM. *Open Source Routing Machine*. Disponível em: <http://project-osrm.org>.
- [4] Streamlit Inc. *Streamlit Documentation*. Disponível em: <https://docs.streamlit.io>.